Supplementary Technical Appendix: Avatar Concession Process in the Bargaining Experiment

1 Overview

This document describes the stochastic process used to generate the **avatar's concession timing** in the interactive bargaining task. The goal of this design is to produce avatar movements that:

1. Follow a predictable expected concession trajectory (deterministic mean behavior), and
2. Exhibit realistic random variation in the timing of concessions, making the avatar appear "alive" while maintaining exogeneity—as concession timing is generated independently of participant behavior.

To achieve this, concessions are modeled as events in a *non-homogeneous Poisson process (NHPP)* whose time-varying rate (intensity) corresponds to the deterministic mean trajectory of an Ornstein–Uhlenbeck process.

Each concession moves the bargaining slider by exactly one discrete step; thus, only the *timing* of events is stochastic.

We generated 500 i.i.d. concession schedules that were drawn randomly for each bargaining round in the avatar-mediated phase.

2 Mean Rate Function

Let $\lambda(t)$ denote the instantaneous expected rate (in concessions per second) of the avatar at time t. We specify $\lambda(t)$ as a monotonically decreasing function:

$$\lambda(t) = \theta + (Y_0 - \theta)e^{-\kappa t}, \tag{1}$$

where:

- θ is the long-run (asymptotic) concession rate,
- Y_0 is the initial concession rate at $t = 0$, and
- $\kappa > 0$ controls how rapidly the rate decays.

This form is equivalent to the mean of an Ornstein–Uhlenbeck process:

$$dY_t = \kappa(\theta - Y_t)\, dt + \sigma\, dW_t, \tag{2}$$

but we use only the deterministic mean trajectory, omitting the diffusion term.

Expected cumulative concessions

The expected number of concessions by time t is:

$$\begin{aligned} \mathbb{E}[N_t] &= \Lambda(t) && (3)\\ &= \int_0^t \lambda(s)\, ds && (4)\\ &= \theta t + \frac{Y_0 - \theta}{\kappa}(1 - e^{-\kappa t}). && (5) \end{aligned}$$

At timeout T, the expected total concessions are:

$$\boxed{\mathbb{E}[N_T] = \theta T + \frac{Y_0 - \theta}{\kappa}(1 - e^{-\kappa T}).} \tag{6}$$

This formula allows calibration of parameters (Y_0, θ, κ) such that the avatar is expected to reach a predefined concession endpoint (e.g., an even split) by time T.

3 Sampling the Concession Times

We model actual concession events as a **non-homogeneous Poisson process (NHPP)** with intensity $\lambda(t)$ given by (1).

3.1 Definition of NHPP

An NHPP $\{N(t)\}_{t\geq 0}$ with rate $\lambda(t)$ satisfies:

1. $N(0) = 0$;
2. Independent increments: disjoint intervals have independent event counts;
3. For small dt,

$$\Pr(N(t+dt) - N(t) = 1) = \lambda(t)\,dt + o(dt), \quad \Pr(N(t+dt) - N(t) \geq 2) = o(dt).$$

3.2 Waiting-time distribution

Given the process is at time t, the probability that no event occurs within the next u seconds is:

$$\Pr(W_t > u) = \exp\Big(- \int_t^{t+u} \lambda(s)\,ds \Big), \tag{7}$$

where W_t is the waiting time to the next concession. The corresponding probability density function is:

$$f_{W_t}(u) = \lambda(t+u)\, \exp\Big(- \int_t^{t+u} \lambda(s)\,ds \Big). \tag{8}$$

Equation (7) generalizes the exponential survival function to a time-varying rate.

4 Ogata Thinning Algorithm

To sample an NHPP efficiently, we use the **Ogata thinning method** (Ogata, 1981).

4.1 Conceptual idea

We simulate a *dominating* homogeneous Poisson process (HPP) with constant rate M such that $M \geq \lambda(t)$ for all $t \in [0, T]$, and then *thin* it by independently accepting each candidate event at time u with probability $\lambda(u)/M$.

The accepted events form an exact NHPP with rate $\lambda(t)$.

4.2 Algorithm

1. Initialize $t \leftarrow 0$, $M \leftarrow \max_{t\in[0,T]} \lambda(t) = Y_0$, and create an empty list $\mathcal{E}$.
2. **Generate a candidate event:** Draw an exponential gap $\Delta \sim \text{Exp}(M)$ and set candidate time $u = t + \Delta$.
3. If $u > T$: stop and return $\mathcal{E}$.
4. Compute the instantaneous rate $\lambda(u)$ using (1).
5. Draw $U \sim \text{Uniform}(0, 1)$. If $U \leq \lambda(u)/M$: **accept** the candidate (append u to $\mathcal{E}$); else **reject**.
6. Set $t \leftarrow u$ and return to step 2.

4.3 Properties

- The accepted event times $\mathcal{E} = \{t_1, t_2, \ldots, t_K\}$ are an exact realization of an NHPP with rate $\lambda(t)$.
- The total count $N_T = K$ is Poisson-distributed with mean $\mathbb{E}[N_T] = \int_0^T \lambda(s)\, ds$.
- Each accepted event triggers exactly one slider step in the avatar's concession motion.

5 Parameter Calibration Summary

Given a desired expected total concessions n^* by timeout T:

$$n^* = \theta T + \frac{Y_0 - \theta}{\kappa}(1 - e^{-\kappa T}), \tag{9}$$

- Solve for Y_0 if κ, θ are predetermined:

$$Y_0 = \theta + \frac{\kappa(n^* - \theta T)}{1 - e^{-\kappa T}};$$

6 Post-sampling filtering

To ensure that all pre-sampled concession schedules generated by the non-homogeneous Poisson process were comparable (balanced) in both timing and overall magnitude, we applied two sequential filters. First, a *shape filter* required that the proportion of concessions within each of five equal-time segments fall within ± 0.6 standard deviations of the *theoretical* binomial expectation, computed as

$$\mathrm{SD}_i = \sqrt{\frac{q_i(1 - q_i)}{n_\star}},$$

where q_i is the model's theoretical segment share and $n_\star = \mathbb{E}[N_T]$ is the expected total number of concessions. Under a nonhomogeneous Poisson process (NHPP), event counts in disjoint time segments are independent Poisson random variables with means proportional to their respective integrated intensities. Conditional on the total event count, these segment counts follow a multinomial distribution with probabilities equal to the relative integrated intensities. Replacing empirical event count with expectation yields a suitable approximation. This criterion enforces fidelity to the model-implied temporal pattern while retaining stochastic variability consistent with the theoretical sampling distribution. Second, a *scale*

gate (truncation) ensured that each schedule reached at least 21 total concessions before timeout ($20 < N_T < 26$), guaranteeing that the avatar always achieved the even-split equilibrium within the bargaining window . Additionally, to avoid unnaturally abrupt starts in the bargaining sequence, each accepted schedule incorporated a small independent random initial delay $D \sim \text{Uniform}(1.5, 3)$ seconds. The inclusion of this adjustment removes, in expectation, approximately $\lambda(T)\mathbb{E}[D] \approx 0.2 \times 2.25 = 0.45$ concessions (less than 2% of total mass, relative to total expected concessions) from the slowest-rate tail near the timeout—a negligible effect that does not distort the overall distribution. Therefore, we refrain from embedding this offset into the data-generating process (DGP) itself, as doing so would require implementing draw-specific CDF computations that add complexity with no meaningful empirical benefit. Together, these design constraints standardize both the temporal structure and the overall intensity of concession behavior across participants. In the implementation, we first add the initial delay to each raw NHPP schedule, and then applied the shape and scale filters.

7 Summary

- Expected concession rate: $\lambda(t) = \theta + (Y_0 - \theta)e^{-\kappa t}$.
- Expected cumulative concessions: $\mathbb{E}[N_t] = \theta t + \frac{Y_0 - \theta}{\kappa}(1 - e^{-\kappa t})$.
- Realized concession times: NHPP with rate $\lambda(t)$, sampled via Ogata thinning.
- Number of events: $N_T \sim \text{Poisson}(\Lambda(T))$.
- Implementation: Generate a fixed list of valid concession schedules, randomly draw one concession schedule for on each bargaining round, trigger concessions following the scheduled timeline.

Parameters used:

- $T = 60$ (Bargain timeout)
- $n^* = \mathbb{E}[N_T] = 23$
- $\theta = 0.2$
- $\kappa = 0.075$
- $Y_0 = 1.03$

Output:

- 500 independent accepted concession schedules from the filtered NHPP-based design.
- Total concessions
  - Mean: 22.76
  - SD: 1.37
  - Max: 25
  - Min: 21
- Time to equilibrium (even split)
  - Mean: 44.95
  - SD: 6.30
  - Max: 58.06
  - Min: 29.91

8 Code

The following document contains all implementation details.

Avatar Concession Schedule Generator

Parameters

```
set.seed(8)

timeout <- 60 # Bargain timeout (bargain window)

n_star <- 23 # Expected number of concessions at timeout

theta <- 0.2 # Ending (terminal) rate of concessions per second of bargaining

# 0.2 = 5 seconds per step

kappa <- 0.075 # Concession rate convergence speed
```

Get Starting Rate

Get Y_0:

$$Y_0 = \theta + \frac{\kappa(n^* - \theta T)}{1 - e^{-\kappa T}}$$

```
# Starting rate of concessions per second of bargaining

Y0 <- theta + (kappa*(n_star - (theta*timeout)))/(1 - exp(-(kappa*timeout)))

Y0
```

```
[1] 1.034268
```

Define Rates

Define rate as a function of time:

$$\lambda(t) = \theta + (Y_0 - \theta)e^{-\kappa t}$$

With CDF equal to the expected number of concessions at time t:

$$\mathbb{E}[N_t] = \Lambda(t) = \int_0^t \lambda(s)\, ds = \theta t + \frac{Y_0 - \theta}{\kappa}(1 - e^{-\kappa t})$$

```
lambda <- function(time) {

  theta + (Y0 - theta) * exp(-(kappa*time))

}

LAMBDA <- function(time) {

  theta * time + ((Y0 - theta) / kappa) * (1 - exp(-kappa * time))

}

# Verify expectations E(N_t) = n_star

LAMBDA(timeout)
```

```
[1] 23
```

```
LAMBDA(timeout) == n_star
```

```
[1] TRUE
```

Define Theoretical Densities

Define Segments

```r
# Split total time in equally spaced segments

segments <- c()

Nsegments <- 5

for (i in 1:(Nsegments+1)) {

  segments[i] <- timeout / Nsegments * (i-1)

}

segments
```

```
[1]  0 12 24 36 48 60
```

Estimate Frequencies and Densities

```r
frequencies <- c()

densities <- c()

for (i in 1:length(segments[-1])) {

  frequencies[i] <- LAMBDA(segments[i+1]) - LAMBDA(segments[i])

  densities[i] <- frequencies[i] / LAMBDA(segments[length(segments)])

}

frequencies
```

```
[1] 9.001065 5.083793 3.491149 2.843628 2.580366
```

```r
sum(frequencies)
```

```
[1] 23
```

```r
densities
```

```
[1] 0.3913507 0.2210345 0.1517891 0.1236360 0.1121898
```

```r
sum(densities)
```

```
[1] 1
```

```r
densitySD <- sqrt(densities*(1-densities)/n_star) # Binomial SD
densitySD
```

```
[1] 0.10176600 0.08652173 0.07481837 0.06863582 0.06580712
```

```r
# Calibrate absolute frequency deviations per segment (allowed ranges)

scalingSD <- 0.6

cat("Maximum allowed frequencies: ", frequencies + densitySD * n_star * scalingSD, "\n\n")
```

```
Maximum allowed frequencies:  10.40544 6.277793 4.523642 3.790802 3.488504
```

```r
cat("Expected frequencies: ", frequencies, "\n\n")
```

```
Expected frequencies:  9.001065 5.083793 3.491149 2.843628 2.580366
```

```r
cat("Minimum allowed frequencies: ", frequencies - densitySD * n_star * scalingSD, "\n\n")
```

```
Minimum allowed frequencies:  7.596694 3.889793 2.458655 1.896454 1.672227
```

```r
cat("Feasible upper: ", floor(frequencies + densitySD * n_star * scalingSD), "\n\n")
```

```
Feasible upper:  10 6 4 3 3
```

```r
cat("Feasible lower: ", ceiling(frequencies - densitySD * n_star * scalingSD), "\n\n")
```

```
Feasible lower:  8 4 3 2 2
```

```
cat("Upper diff: ", floor(frequencies + densitySD * n_star * scalingSD) - frequencies, "\n\n
```

```
Upper diff:  0.998935 0.9162073 0.5088513 0.156372 0.4196343
```

```
cat("Lower diff: ", frequencies - ceiling(frequencies - densitySD * n_star * scalingSD), "\n
```

```
Lower diff:  1.001065 1.083793 0.4911487 0.843628 0.5803657
```

Empirical

```
get_empirical <- function(schedule, segments) {

  emprirical <- c()

  for (i in segments) {
    density <- sum(schedule <= i) / length(schedule)

    emprirical <- c(emprirical, density)
  }

  error <- abs(densities - diff(c(0, emprirical)))

  return(list(error=error, density=diff(c(0, emprirical))))
}
```

Draw Samples

NHPP with Ogata Thinning

Sample from a non-homogenous poisson process using Ogata thinning algorithm:

```
rejectionCount <- 0 # Discarded draws, save for diagnostics

scheduleList <- list()

densityMatrix <- matrix(nrow=0, ncol=Nsegments) # Save for diagnostics

# Get upper bound rate, include rate at time=0 in case rate is decreasing
```

```
M <- max(lambda(timeout), lambda(0), theta)

while (length(scheduleList) < 500) {

  t <- 0

  schedule <- c()

  while (t < timeout) {

  interval <- rexp(1, rate=M)

  nextTime <- t + interval

  if (nextTime > timeout) break

  if (runif(1) <= lambda(nextTime)/M) {

    schedule <- c(schedule, nextTime)

  }

  t <- nextTime

  }

  # Add a random initial delay to avoid jumpy start to the bargain

  schedule <- schedule + runif(1, min=1.5, max=3)

  # Filter out of range draws (after adding extra initial delay)

  schedule <- schedule[schedule < timeout]

  # Filter draws that deviate from the theoretical density

  empirical <- get_empirical(schedule, segments[-1])

  distributionFilter <- sum(empirical$error > scalingSD*densitySD) == 0

  # Filter draws that deviate from the expected count (even split equilibrium ensurance)
```

```
  scheduleLengthFilter <- 20 < length(schedule) & length(schedule) < 26
  if (distributionFilter & scheduleLengthFilter) {
    scheduleList[[length(scheduleList) + 1L]] <- schedule
    densityMatrix <- rbind(densityMatrix, empirical$density)
  } else {
    rejectionCount <- rejectionCount + 1
  }
}
```

```
#saveRDS(scheduleList, "ConcessionScheduleList.rds")
```

Check Output

Check output is well behaved

```
cat("Number of rejected draws: ", rejectionCount, "\n\n")
```

```
Number of rejected draws:  37973
```

```
cat("Theoretical densities: ", densities, "\n\n")
```

```
Theoretical densities:  0.3913507 0.2210345 0.1517891 0.123636 0.1121898
```

```
cat("Empirical (mean) densities: ", colMeans(densityMatrix), "\n\n")
```

```
Empirical (mean) densities:  0.3904929 0.224673 0.1558537 0.1175437 0.1114367
```

```
cat("Theoretical SDs: ", densitySD, "\n\n")
```

```
Theoretical SDs:  0.101766 0.08652173 0.07481837 0.06863582 0.06580712
```

```r
cat("Empirical SDs: ", apply(densityMatrix, 2, sd), "\n\n")
```

```
Empirical SDs:  0.03127602 0.02895061 0.02291162 0.02415957 0.02288469
```

```r
cat("Empirical max: ", apply(densityMatrix, 2, max), "\n\n")
```

```
Empirical max:  0.44 0.2727273 0.1904762 0.16 0.1428571
```

```r
cat("Empirical min: ", apply(densityMatrix, 2, min), "\n\n")
```

```
Empirical min:  0.3333333 0.173913 0.12 0.08333333 0.08
```

```r
cat("Empirical range: ", apply(densityMatrix, 2, max) - apply(densityMatrix, 2, min), "\n\n")
```

```
Empirical range:  0.1066667 0.09881423 0.07047619 0.07666667 0.06285714
```

```r
evenSplitTime <- c()

totalConcessions <- c()


for (i in scheduleList) {

  evenSplitTime <- c(evenSplitTime, i[20])

  totalConcessions <- c(totalConcessions, length(i))

}

cat("Even split time, Mean: ", mean(evenSplitTime), " SD: ", sd(evenSplitTime), " Max: ", max
```

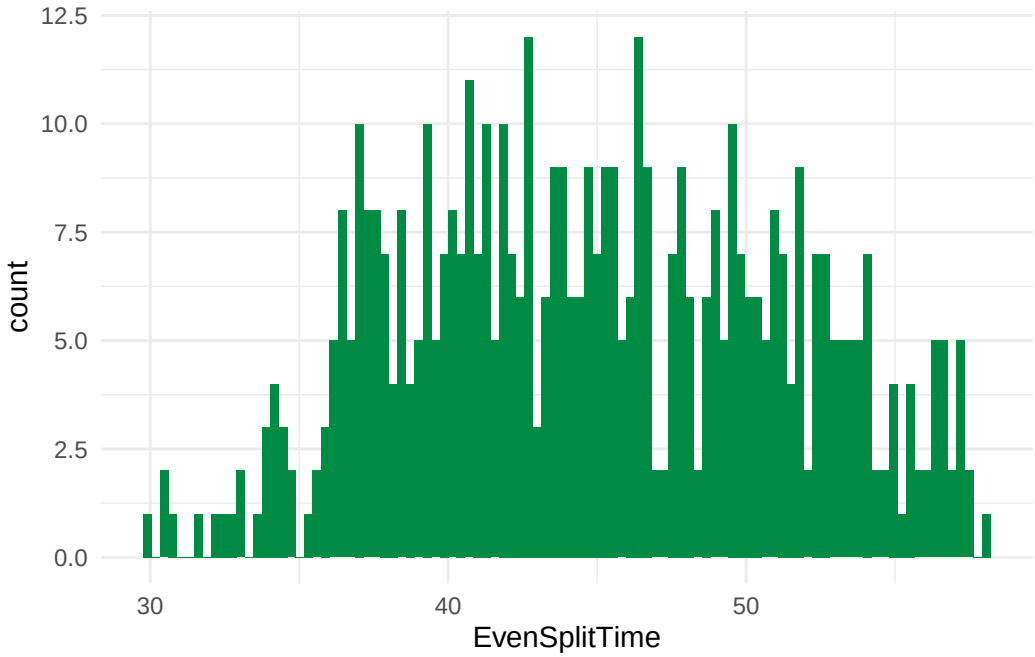
```
Even split time, Mean:  44.95284  SD:  6.297667  Max:  58.06332  Min:  29.90604
```

```r
cat("Total concessions, Mean: ", mean(totalConcessions), " SD: ", sd(totalConcessions), " Max
```

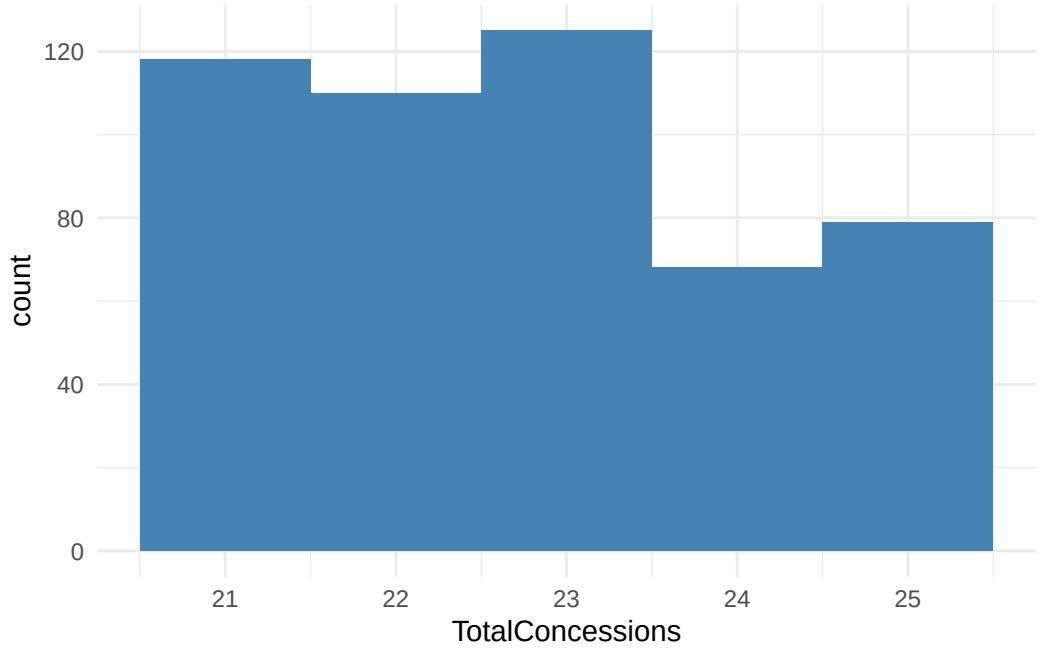
```
Total concessions, Mean:  22.76  SD:  1.370458  Max:  25  Min:  21
```

```r
df <- data.frame(EvenSplitTime = evenSplitTime, TotalConcessions = totalConcessions)

ggplot(df, aes(x=EvenSplitTime))+
  geom_histogram(bins=100, fill="springgreen4")+
  theme_minimal()
```




```r
ggplot(df, aes(x=TotalConcessions))+
  geom_histogram(bins=5, fill="steelblue")+
  theme_minimal()
```



Export to JSON

```
# Quantize helper (~float32-ish precision in seconds)
#q <- function(x, digits = 4) round(as.numeric(x), digits)

#schedules_json <- lapply(seq_along(scheduleList), function(i) {
#  list(
#    totalConcessions = totalConcessions[i],
#    timeToEquilibrium = evenSplitTime[i],
#    timestamps = q(scheduleList[[i]], 4)  # e.g., 12.3456
#  )
#})

#write_json(
#  schedules_json,
#  path = "ConcessionSchedules.json",
#  auto_unbox = TRUE,
#  pretty = FALSE
#)
```